browser clients and pure-JavaScript API clients this format offers similar advantages to using the fragment but with the added advantage that this part of the URI is explicitly designed for holding credentials and so can be easily recognized by clients. Listing 9.9 shows how to parse and load a capability URI in this format from a JavaScript API client. It first parses the URI using the `URL` class and extracts the token from the `password` field. Then you should remove the username field and instead copy the token into the Authorization header before making a `fetch` request to the API. Navigate to src/main/resources/public and create a new file named capability.js with the contents of the listing.

Listing 9.9 Loading a capability URI from JavaScript

```
function getCap(url, callback) {
let capUrl = new URL(url); #A
let token = capUrl.password; #A
capUrl.password = ''; #B
return fetch(capUrl.href, { #C
headers { 'Authorization': 'Bearer ' + token } #C
})
.then(response => response.json())
.then(callback)
.catch(err => console.error('Error: ', err));
}
```

#A Parse the URL and extract the token from the username component.
#B Blank out the username component.
#C Now fetch the URI to call the API with the token.